
***FALAT*: Tracing Failures in LLM Agent Trajectories via Dependency-Guided Search**

Md Nakhla Rafi
 SPEAR Lab
 Concordia University
 Montreal, Canada
 mdnakhla.rafi@mail.concordia.ca

Md Ahasanuzzaman
 SPEAR Lab
 Concordia University
 Montreal, Canada
 m_ahasa@live.concordia.ca

Dong Jae Kim
 DePaul University
 Chicago, USA
 dkim121@depaul.edu

Zhijie Wang
 Concordia University
 Montreal, Canada
 zhijie.wang@concordia.ca

Tse-Hsun Chen
 SPEAR Lab
 Concordia University
 Montreal, Canada
 peterc@encs.concordia.ca

Abstract

LLM-based agents increasingly solve complex tasks through long trajectories involving reasoning steps, tool calls, and inter-agent communication. However, when these agents fail, it is often unclear which agent caused the failure and which step introduced the decisive error. This attribution problem is challenging because mistakes can propagate across the trajectory: later actions may appear incorrect, but only because they depend on an earlier corrupted state. Therefore, failure attribution cannot be treated as independent step-level classification. We propose *FALAT*, a diagnostic framework for failure attribution in LLM agent trajectories. *FALAT* frames attribution as a dependency-guided search problem. It first constructs an expectation of how the task should be solved and uses this expectation to identify suspicious regions in the trajectory. It then traces dependencies among decisions, tool outputs, and agent messages to distinguish error-introducing steps from steps that merely inherit or propagate prior mistakes. Finally, *FALAT* evaluates whether correcting a candidate step would be sufficient to recover the expected outcome, allowing it to identify both the responsible agent and the decisive failure step. We evaluate *FALAT* on the Who&When benchmark, which includes both algorithm-generated and hand-crafted multi-agent failure trajectories. The results show that *FALAT* consistently improves responsible-agent and decisive-step attribution. Its best configurations achieve 46.0% step-level accuracy on algorithm-generated trajectories and 29.1% on the more challenging hand-crafted trajectories, outperforming specialized attribution baselines and direct prompting with standalone LLMs. These findings suggest that dependency-aware reasoning is essential for reliable failure diagnosis in LLM agent systems.

1 Introduction

Large language models (LLMs) have enabled a new class of agentic systems that solve complex tasks through iterative reasoning, tool use, and environment interaction. Modern software engineering agents such as OpenAI Codex and Claude Code can autonomously edit code, invoke tools and

scripts, and execute tests over long execution horizons. Such a sequence of actions generated and executed by these agents is also referred to as a *trajectory*.

Despite rapid progress, state-of-the-art (SOTA) agents still fall short on many real-world tasks, particularly those involving long-range dependencies, under-specified requirements, and multi-step execution chains [Jimenez et al., 2023, Yu et al., 2025]. Understanding *why* an agent failed is thus increasingly important as these systems become more autonomous. When a trajectory produces an incorrect output, developers must determine which agent (*who*) introduced the decisive error at which step(s) (*when*), where a decisive error refers to an error whose correction would recover the final outcome of the agentic system.

Addressing this problem, which we refer to as *failure attribution*, is essential for ensuring the reliability of agentic systems. However, decisive error localization is challenging because trajectories are often long, interdependent, and difficult to inspect directly. Moreover, once an early mistake corrupts the execution state, many downstream steps may appear locally reasonable while still propagating the original fault. As a result, the true error source can be obscured by later symptoms.

Recent failure-attribution methods address these issues only partially. Single-pass LLM judges treat trajectories as flattened inputs, making them vulnerable to positional bias [Liu et al., 2024], order sensitivity in the presented evidence [Rafi et al., 2026], and limited hypothesis revision across long chains [Zhu et al., 2025]. Learning-based tracers [Zhang et al., 2025a] independently evaluate candidate steps and predict which one is responsible for the failure. While intuitive, this formulation generalizes poorly in practice [Zainullina et al., 2025].

In this paper, we reformulate failure attribution as *hierarchical dependency-guided search*. Rather than classifying steps independently, *FALAT* performs a top-down search across layers of abstraction and uses typed dependencies to guide error candidate pruning and verification. *FALAT* first constructs a task-conditioned prior and a hierarchical representation of the trajectory to support coarse-to-fine localization. The prior is constructed independently of the trajectory’s intermediate steps so that candidate selection is not guided solely by the agent’s own flawed reasoning. *FALAT* then constructs typed dependencies among candidate steps to separate possible error sources from downstream carriers and prune candidates whose effects cannot reach the final output. The remaining candidates are ranked and verified by checking whether fixing a candidate would recover the expected outcome. Finally, *FALAT* performs local verification around the predicted decisive step to reduce premature commitment to a plausible but non-decisive candidate.

We evaluate *FALAT* on the Who&When benchmark [Zhang et al., 2025b] using three LLM backbones and compare it against two state-of-the-art attribution baselines and multiple standalone LLM baselines. *FALAT* consistently achieves higher responsible-agent and decisive-step attribution accuracy. On algorithm-generated trajectories, *FALAT* reaches 46.0 step-level accuracy, compared with 37.3 for CHIEF [Wang et al., 2026] under the same MiniMax 2.5 [MiniMax AI, 2025] backbone. On the more challenging hand-crafted trajectories, *FALAT* reaches 29.1 step-level accuracy, compared with 17.0 for CHIEF under DeepSeek V3.2 [Liu et al., 2025].

Our contributions are summarized as follows:

- We formulate failure attribution in LLM agent trajectories as *hierarchical dependency-guided search*, shifting the task from flat step classification to structured diagnosis over candidate steps and their dependencies.
- We propose *FALAT*, a diagnostic framework that uses expected task behavior and step dependencies to identify where an error is introduced.
- We evaluate *FALAT* on the Who&When benchmark across multiple LLM backbones, showing consistent gains over specialized attribution baselines and direct prompting with standalone LLMs, especially on the stricter step-level attribution task.

2 Hierarchical Dependency-Guided Search for Agent Trajectory Diagnostics

In this paper, we reformulate failure attribution as *hierarchical dependency-guided search*. The task is to identify the earliest step, or set of steps, whose correction would change the system’s final output from the observed incorrect output $\hat{o}$ to the expected output o^* . The search proceeds top-down through layers of abstraction and uses typed dependency relations to prune candidates.

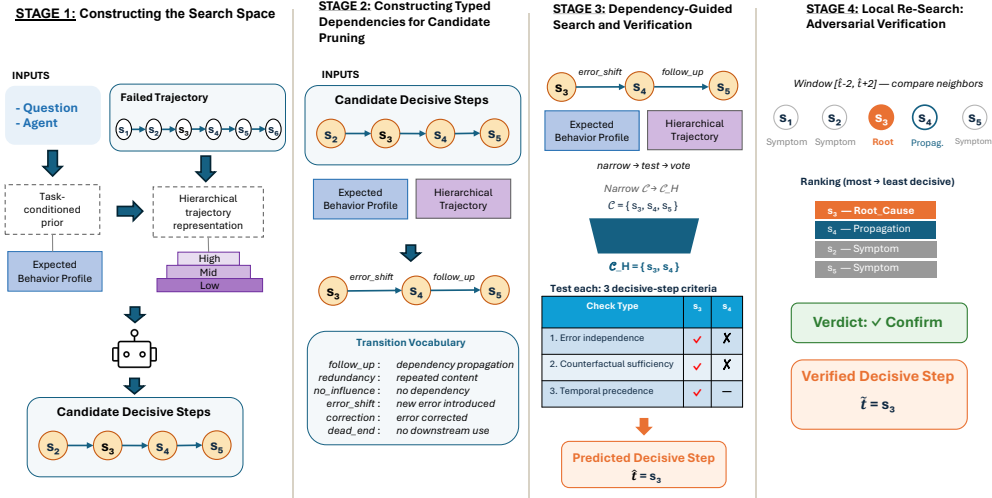


Figure 1: Overview of *FALAT*. Stage 1 constructs an external prior π , a three-level trajectory representation M , and an initial candidate set C . Stage 2 constructs typed dependencies to separate possible error sources from downstream carriers and prune candidates. Stage 3 performs dependency-guided search and verifies whether fixing a candidate would recover the expected output. Stage 4 locally verifies the predicted decisive step.

This reformulation addresses two properties of the problem that step-level classification ignores. First, directly searching on trajectories spanning hundreds of steps can be both inefficient and ineffective compared to coarse-to-fine narrowing, as shown in classical abstraction hierarchies for planning [Sacerdoti, 1974] and reinforcement learning [Sutton et al., 1999]. Second, since errors may propagate across steps, distinguishing origination from propagation requires *deductive* reasoning over dependency structure rather than ranking individual steps independently.

These observations motivate three design principles for *FALAT*.

- **Expectation-guided search.** Candidate error steps should be identified using an external prior over expected behavior. Signals derived solely from the trajectory can be misleading because they may reflect the same flawed assumptions that led the agent to fail.
- **Source-propagation separation.** Dependency relations between candidate steps should be typed to distinguish error origination from propagation. Without this distinction, straightforward LLM judgments often mistake downstream propagated errors for decisive causes [Guo et al., 2026].
- **Counterfactual sufficiency.** A candidate is decisive only if fixing it would recover the expected output o^* . This counterfactual test holds earlier steps unchanged while allowing later steps to adjust, filtering out visible errors that do not affect the final outcome.

FALAT operationalizes these principles as a four-stage search procedure (Figure 1). The first stage **constructs the search space**: a task-conditioned prior π and a hierarchical trajectory representation M define what is searched and what counts as a deviation. The second stage **constructs typed dependencies for candidate pruning**: typed transitions determine which candidates can be pruned and which remain as possible error sources. The third stage **executes dependency-guided search**: the candidate set is narrowed and checked by asking whether fixing a candidate would recover the expected output. The fourth stage performs **local verification**: nearby candidates are re-examined to reduce premature commitment to a plausible but non-decisive step.

Notation. Let $\mathcal{G}$ denote an agentic system and Q denote a task specification. A trajectory $\tau = (s_1, s_2, \dots, s_T)$ is an ordered sequence of T steps generated by $\mathcal{G}$ in response to Q . Each step $s_i = (r_i, u_i, o_i)$ consists of intermediate reasoning r_i , an action or message u_i , and the resulting output or observation o_i . A trajectory is classified as a failure if the final output $\hat{o}$ produced at s_T differs from the expected output o^* , i.e., $\hat{o} \neq o^*$. A step s_i is *active* if it involves an independent decision, such as retrieval, computation, filtering, tool use, or code execution, and *passive* if it only relays prior information. Let $\mathcal{I}_{\text{act}}(\tau) \subseteq \{1, \dots, T\}$ denote the indices of active steps in τ .

Definition (decisive step set). We formalize failure diagnosis as identifying a minimal subset of active steps whose correction would be sufficient to recover the expected outcome. Formally, a decisive step set S^* satisfies

$$S^* \in \arg \min_{S \subseteq \mathcal{I}_{\text{act}}(\tau)} |S| \quad \text{s.t.} \quad \exists \{s'_t\}_{t \in S} : f(\tau^{(S \leftarrow S')}) = o^*.$$

Here, $\tau^{(S \leftarrow S')}$ denotes the counterfactual trajectory obtained by replacing each selected step s_t with a corrected step s'_t , while allowing subsequent steps to adapt to the corrected state. The function f returns the final output induced by this counterfactual trajectory. When the minimal set contains a single step, the decisive index is $t^* = \min S^*$, which recovers the standard step-level formulation [Zhang et al., 2025b]. Throughout the paper, the search target is S^* and the responsible agent(s) associated with its steps, denoted by α^* . For exposition, and to match the benchmark annotation format, we focus on the common single-step case unless otherwise noted.

2.1 Constructing the Search Space

Effective searching in long trajectories requires a structured space in which coarse regions can be examined and pruned before fine-grained inspection.

Defining an external prior. We construct the external π from the task specification Q , the agent set $\mathcal{A}$, the agents' roles and system prompts $\mathcal{P}$, the available tools $\mathcal{T}$, and the failed final output $\hat{o}$. The intermediate reasoning, actions, or tool outputs in τ are not used to ensure π is independent of any intermediate steps. This independence is crucial because agentic failures can remain internally coherent: once an early error corrupts the state, later steps may be reasonable responses to that corrupted state. Evaluators based only on trajectory self-consistency may therefore miss the original deviation. The prior records the task objective, expected output type, appropriate agent/tool use at each stage, tool-use constraints, and task-specific failure risks such as unsupported information use, skipped verification, or incorrect intermediate values [Bouzenia and Pradel, 2025]. These risks serve only as guidance for identifying suspicious steps, not as evidence of failure.

Hierarchical representation as a search space. Rather than searching directly over τ , we construct a hierarchical abstraction that reduces the search space. Specifically, we convert τ into a three-level representation $M = (M_{\text{hi}}, M_{\text{mid}}, M_{\text{lo}})$ (Table 1), conditioned on the external prior π . The high level captures coarse execution stages, the mid level summarizes local decisions and their significance, and the low level preserves raw execution evidence such as tool outputs and concrete values. This coarse-to-fine decomposition follows the principle of abstraction hierarchies in planning [Sacerdoti, 1974, Miki et al., 2015] and temporal abstraction in reinforcement learning [Sutton et al., 1999]. It allows search to first identify suspect stages in M_{hi} , then localize candidate steps in M_{mid} , and finally verify them against evidence in M_{lo} .

FALAT constructs a hierarchical representation M from a trajectory $\tau = (s_1, \dots, s_T)$ via non-overlapping window-based abstraction. The trajectory is partitioned into $N = \lceil T/k \rceil$ consecutive windows $\{W_j\}_{j=1}^N$, where each W_j contains k steps, except possibly the final window, and is processed independently to ensure uniform coverage and reduce positional bias [Liu et al., 2024, Rafi et al., 2026].

For each window W_j , *FALAT* retains the original steps as $M_{\text{lo}}^{(j)} = W_j$, summarizes them into mid-level representations $M_{\text{mid}}^{(j)}$, and further abstracts them into high-level representations $M_{\text{hi}}^{(j)}$. Both abstraction stages are conditioned on the prior π , so actions and decisions are interpreted relative to the expected task behavior rather than only their local text. The full representation is $M = \{(M_{\text{lo}}^{(j)}, M_{\text{mid}}^{(j)}, M_{\text{hi}}^{(j)})\}_{j=1}^N$. Here, π guides abstraction, while the evidence stored in M remains

Level	Contents	Role in Search
High (M_{hi})	Global execution summary: task objective, expected output type, high-level failure pattern, and agent responsibilities	Identifies suspect regions of the trajectory and provides global context for dependency-guided pruning (§2.1)
Mid (M_{mid})	Local step summaries: step index, agent, action, and decision, and significance in the trajectory	Localizes suspicious steps within suspect regions and provides candidate-level context for constructing typed dependencies (§2.2)
Low (M_{lo})	Step-level evidence: original step content, tool outputs, concrete values, assumptions, and provenance	Supports candidate verification against raw execution evidence (§2.3)

Table 1: Hierarchical trajectory representation as a three-level search space. Each level supports a distinct search operation, and the levels are aligned so that coarse-level narrowing constrains lower-level candidate selection and verification.

derived from the observed trajectory. The three levels are aligned, enabling top-down localization from coarse segments in M_{hi} to raw evidence in M_{lo} . Processing windows independently trades limited cross-window context for substantially reduced positional bias in long trajectories.

Error candidate set selection. Using the prior π and the hierarchical representation M , *FALAT* identifies an initial error candidate set $\mathcal{C}$ of possible failure locations. The role of M is to make candidate selection coarse-to-fine rather than purely step-wise: M_{hi} identifies execution stages that deviate from the expected task progress, M_{mid} identifies suspicious steps within those stages by summarizing each step’s agent, action, decision, output, and role in the local context and M_{lo} provides the raw evidence used to check whether the corresponding steps should enter the candidate set. Thus, a step s_i is included in $\mathcal{C}$ if the aligned representations around s_i show at least one signal of potential failure. At this stage, $\mathcal{C}$ is not treated as the decisive set; it only defines the initial search candidates for later typed-transition analysis and counterfactual verification.

Let $\tau = (s_1, \dots, s_T)$ and let $Y(\tau)$ denote the final outcome. For each step s_i , *FALAT* evaluates three signals using the aligned views of M : (i) *outcome relevance*, where M_{hi} and M_{mid} indicate that s_i belongs to a stage or transition that contributes to $Y(\tau)$; (ii) *prior inconsistency*, where the action, decision, or transition associated with s_i conflicts with the task objective, agent role, tool-use constraints, or expected verification behavior encoded in π ; and (iii) *local execution inconsistency*, where M_{lo} shows that the agent’s stated reasoning, executed action, and observed output do not align, such as when an agent calls a tool inconsistent with its stated intent, uses mismatched tool arguments, ignores a tool error, or continues from an empty or unsupported result.

Let $\delta_i^{(r)}, \delta_i^{(p)}, \delta_i^{(e)} \in \{0, 1\}$ denote indicators for these conditions. The candidate set is then

$$\mathcal{C} = \{i \mid \delta_i^{(r)} \vee \delta_i^{(p)} \vee \delta_i^{(e)} = 1\}.$$

Because M is constructed over disjoint windows $\{W_j\}_{j=1}^N$, candidates are consolidated across windows by merging adjacent indices into contiguous regions $\mathcal{R} = \{R_k\}$, where each $R_k \subseteq \{1, \dots, T\}$ is a maximal interval of candidate steps. This consolidation uses the alignment between M_{hi} , M_{mid} , and M_{lo} to recover context when suspicious behavior spans window boundaries.

2.2 Constructing Typed Dependencies for Candidate Pruning

The error candidate set $\mathcal{C}$ identifies steps where deviations may occur, but it does not distinguish between steps that *originate* an error and those that merely *propagate* it. This distinction is critical, as downstream steps often remain internally consistent with earlier errors and are thus incorrectly identified as decisive [Zhang et al., 2025b].

To separate error sources from downstream carriers and prune candidates whose effects cannot reach the final output, we construct a directed dependency structure over candidate steps. For each ordered pair (s_i, s_j) with $i < j$, we assign a transition label $\ell_{i \rightarrow j} \in \mathcal{L}$, where $\mathcal{L}$ is a finite set of diagnostic relation types. These labels describe how information, decisions, and errors move between candidate steps. The resulting structure is used to retain candidates that may introduce or carry a deviation

toward the final output $\hat{o}$, and to prune candidates whose effects are redundant, corrected, unrelated, or unable to reach the final output.

We use the following transition labels:

- **follow_up**: s_j builds on the output, decision, or assumption introduced by s_i . If s_i is aligned with the prior π , the transition indicates normal progression and can be pruned. If s_i is already corrupted, s_j is treated as a downstream carrier, and attribution is traced back to s_i .
- **redundancy**: s_j repeats, restates, or rechecks information already provided by s_i without adding new evidence or changing the trajectory. Such transitions are pruned unless the repeated step introduces an independent deviation.
- **no_influence**: s_j does not consume or depend on the output of s_i . This relation prevents attribution from being transferred across unrelated steps. Candidates connected only through such transitions are pruned unless they independently deviate from the prior.
- **error_shift**: s_j introduces a new deviation, shifts the trajectory away from the expected path, or transforms an upstream issue into a different downstream error. Such steps are retained as possible error sources or error-shifting points.
- **correction**: s_j fixes, compensates for, or invalidates a deviation introduced by s_i . Upstream candidates whose effects no longer propagate beyond the correction are pruned.
- **dead_end**: s_i leads to an output that is not consumed by later reasoning or the final answer. Such candidates are pruned unless there is independent evidence that they affect the final output.

Applying these labels yields a refined candidate set $\mathcal{C}' \subseteq \mathcal{C}$ containing candidates whose effects can plausibly reach the final output $\hat{o}$. This reduces the search space while preserving candidates that may have originated, propagated, or transformed the failure.

2.3 Dependency-Guided Search and Verification

After typed-transition structure, *FALAT* further narrows $\mathcal{C}'$ to a small high-confidence set $\mathcal{C}_H$ and verifies each remaining candidate against the counterfactual sufficiency objective.

Narrowing the candidates. Ranking combines π with the typed transition structure of $\mathcal{C}'$. Candidates are prioritized if they introduce a deviation from π and lie on a propagation path that reaches the final output. They are deprioritized if their outputs are unused, corrected, or independent of the final output (i.e., already pruned by **dead_end**, **redundancy**, or **follow_up** deductions). The remaining candidates are ranked using three signals: (i) whether the step makes an incorrect execution decision (e.g., wrong tool, flawed code, or misused data); (ii) whether it introduces or relies on unverified or fabricated information relative to π ; and (iii) whether it lies on a typed transition path that influences subsequent reasoning or the final output. We retain the top K candidates as the high-confidence set $\mathcal{C}_H$, with $K = 3$ in our experiments. If fewer than K candidates remain after pruning, all remaining candidates are included.

Verification against the counterfactual objective. For each candidate in $\mathcal{C}_H$, *FALAT* constructs a localized verification context centered on the candidate: the prior π , the raw window (r, a, o) for steps $[i-1, i, i+1]$, the typed transitions involving i , and linked M_{i0} evidence. Where available, raw logs such as executable code and full tool outputs replace summarized content to preserve precise execution details. The full representation M remains available when broader context is required.

We then evaluate each candidate using three criteria aligned with the search objective:

1. **Error independence.** Does the step introduce the error, or merely inherit it from an upstream fault? We assess this by tracing typed dependencies backward from the candidate: **error_shift** indicates that the candidate may introduce or transform an error, while **follow_up** indicates propagation from an earlier step.
2. **Counterfactual sufficiency.** Would correcting this step’s output, decision, or assumption recover o^* while keeping prior steps fixed and allowing subsequent steps to adapt?

3. **Temporal precedence.** Among candidates satisfying the above criteria, select the earliest.

The candidate that satisfies all three criteria is selected as the predicted decisive step $\hat{t}$, and its corresponding agent is denoted by $\hat{\alpha}$.

2.4 Local Re-Search: Adversarial Verification

A common issue in LLM-as-judge settings is *anchoring*. Namely, once a plausible candidate is selected, later reasoning may rationalize it rather than challenge it. Thus, the predicted decisive step $\hat{t}$ may still be worse than a nearby alternative under direct comparison. We address this with **local verification**: a bounded re-evaluation that compares $\hat{t}$ against its immediate neighbors and re-applies the source–propagation distinction at finer scope.

Given $\hat{t}$, *FALAT* extracts a bounded neighborhood $[\max(1, \hat{t} - 2), \min(\hat{t} + 2, T)]$ and provides the LLM with π , the relevant typed transitions, and M . For each step in the window, the LLM constructs arguments for and against the step being decisive and assigns one of four roles: **ROOT_CAUSE**, which independently introduces the decisive error; **PROPAGATION**, which carries an upstream error into downstream reasoning; **SYMPTOM**, which reflects an existing error without materially affecting downstream computation; or **CONTRIBUTING**, which affects execution but is not by itself sufficient to cause the final failure.

The four-role taxonomy is the local-search counterpart of the typed-transition vocabulary: **ROOT_CAUSE** and **PROPAGATION** are the cause-vs-carrier distinction restated as step-level roles, while **SYMPTOM** and **CONTRIBUTING** catch the two ways a step can look wrong without being decisive. *FALAT* then ranks the steps in the window from most to least likely to be the decisive error using the role classification, the for/against arguments, the typed transition structure, and a counterfactual check on whether correcting the step would change the final output. This evaluation is comparative by construction since $\hat{t}$ must outperform specific named alternatives, which is precisely what local search requires to escape an anchored local optimum.

FALAT then either *confirms* that $\hat{t}$ remains the highest ranked decisive step or *replaces* it with a higher ranked neighboring step. We denote the verified decisive step $\tilde{t}$ and its associated agent $\tilde{\alpha}$.

3 Experiment Setup

Benchmark. We evaluate *FALAT* on Who&When [Zhang et al., 2025b], a benchmark of 184 failed executions from 127 LLM-based multi-agent systems. The benchmark covers both CaptainAgent-generated teams on GAIA and AssistantBench tasks [Song et al., 2024, Mialon et al., 2023, Yoran et al., 2024] and hand-crafted Magentic-One systems [Fourney et al., 2024]. Each instance provides the full trajectory and human annotations for the responsible agent and decisive error step.

Metrics. Following prior work [Zhang et al., 2025b], we evaluate *FALAT* using three metrics. **Agent-level accuracy** measures whether the predicted responsible agent $\hat{\alpha}$ matches the ground-truth agent α^* . **Step-level accuracy** measures whether the predicted decisive step $\hat{t}$ exactly matches the ground-truth step t^* , making it the strictest metric. **Step-level accuracy with tolerance** considers a prediction correct if $|\hat{t} - t^*| \leq k$. We report results for $k \in \{1, 2, 3, 4, 5\}$.

Baselines. We compare *FALAT* with CHIEF [Wang et al., 2026], which we re-run under the same three backbone models as *FALAT*, and AgentTracer-8B [Zhang et al., 2025a], whose results are adopted from prior work. We also include direct-prompting LLM baselines from Who&When, where the full trajectory is provided to the model in a single pass.

4 Result

Overall performance. *FALAT* achieves the strongest results on both trajectory types (Table 2). Its best configuration reaches 68.0 agent-level and 46.0 step-level accuracy on algorithm-generated trajectories, and 73.0 agent-level and 29.1 step-level accuracy on hand-crafted trajectories. Compared with CHIEF and AgentTracer, *FALAT* shows the largest gains at the step level, the stricter and more diagnostic metric. Against CHIEF, *FALAT* improves agent-level accuracy by up to +13.2 points (MiniMax 2.5) on algorithm-generated trajectories and +23.0 points (DeepSeek V3.2) on

Table 2: Performance on algorithm-generated and hand-crafted trajectories. We report agent-level accuracy (Agent) and exact step-level accuracy (Step). Best results within each block are bolded. For fair comparison, we re-run CHIEF under the same backbone configurations used by *FALAT*.

Backbone / Model	Method	Algorithm-Generated ($n = 126$)		Hand-Crafted ($n = 58$)	
		Agent	Step	Agent	Step
Same-backbone comparison					
GPT-5.4-Mini	CHIEF	59.0	27.0	55.2	13.8
	FALAT	66.5	42.0	65.5	22.2
DeepSeek V3.2	CHIEF	52.0	30.0	50.0	10.3
	FALAT	59.7	38.0	73.0	29.1
MiniMax 2.5	CHIEF	54.8	37.3	51.0	17.0
	FALAT	68.0	46.0	66.3	25.9
Specialized attribution baseline (results adopted from Zhang et al. [2025a])					
AgentTracer-8B	AgentTracer	63.7	37.3	63.8	20.7
Standalone LLM baselines (results adopted from Zhang et al. [2025a])					
Qwen3-8B	Direct	60.3	5.6	39.5	3.5
LLaMA-3.2-3B	Direct	45.2	8.7	50.0	3.5
Qwen3-32B	Direct	57.9	8.7	44.8	1.7
Qwen3-Coder	Direct	36.5	32.5	60.4	13.8
GPT-4.1	Direct	59.5	21.9	37.9	3.4
DeepSeek-R1	Direct	65.1	29.5	53.4	6.9
Gemini-2.5-Pro	Direct	57.1	25.9	51.7	6.9
Claude-Sonnet-4	Direct	51.1	38.8	50.0	18.9

Table 3: Step-level accuracy under tolerance windows on Hand-Crafted trajectories. We report whether the predicted decisive step falls within k steps of the ground-truth step ($|\hat{t} - t^*| \leq k$).

Tolerance	Hand-Crafted		
	GPT-5.4-Mini	DeepSeek V3.2	MiniMax 2.5
Exact	22.2	29.1	25.9
± 1	22.4	32.5	32.8
± 2	25.9	36.7	34.5
± 3	32.8	40.0	37.9
± 4	41.4	45.6	51.7
± 5	43.1	49.1	53.4

hand-crafted trajectories. At the step level, *FALAT* improves performance by +8.7 points (MiniMax 2.5) on algorithm-generated trajectories and +12.1 points (DeepSeek V3.2) on hand-crafted trajectories. Compared to AgentTracer, *FALAT* further improves step-level accuracy by +8.7 points on algorithm-generated trajectories and +5.2 points on hand-crafted trajectories. These results indicate that *FALAT* provides a more effective attribution procedure than existing trajectory-level baselines, especially for the harder step-localization task.

Comparison to standalone LLMs. Increasing model scale alone does not reliably solve step-level attribution. Small standalone models achieve near-zero step-level accuracy, while medium and large models remain unstable across trajectory types. The strongest standalone baseline, Claude-Sonnet-4, reaches 38.8 step-level accuracy on algorithm-generated trajectories but only 18.9 on hand-crafted trajectories. In contrast, *FALAT* improves step-level accuracy by +7.2 points (38.8 $\rightarrow$ 46.0) on algorithm-generated trajectories and by +10.2 points (18.9 $\rightarrow$ 29.1) on hand-crafted trajectories. These results suggest that stronger backbones alone are insufficient for reliable step-level attribution, and that structured search provides additional benefits beyond model scaling.

Performance under tolerance windows. In practice, precisely identifying the decisive step is not always required; narrowing the failure to a range of steps is often sufficient. We therefore report tolerance-based step-level performance of *FALAT* in Table 3, focusing on hand-crafted trajectories due to their higher difficulty. Algorithm-generated trajectories are excluded, as their short length (maximum of 10 steps) would lead to inflated accuracy under relaxed tolerances [Zhang et al.,

Table 4: Ablation results on algorithm-generated and hand-crafted trajectories. All experiments use GPT-5.4-Mini as the backbone.

Configuration	Algorithm-Generated		Hand-Crafted	
	Agent	Step	Agent	Step
Full <i>FALAT</i>	66.5	42.0	65.5	22.2
No external prior	55.6 (-10.9)	36.5 (-5.5)	58.6 (-6.9)	22.4 (+0.2)
No hierarchical representation	50.0 (-16.5)	31.7 (-10.3)	53.8 (-11.7)	17.2 (-5.0)
No transition typing	57.9 (-8.6)	37.3 (-4.7)	55.0 (-10.5)	18.8 (-3.4)

2025b]. As shown in Table 3 exact step-level accuracy ($k = 0$) remains modest (22.2–29.1), but increases steadily with relaxed tolerance, reaching up to 53.4 at ± 5 . This indicates that even when *FALAT* does not predict the exact step, it typically localizes the error within a small neighborhood of the ground-truth decisive step.

Ablation study. Table 4 shows how each component contributes to *FALAT*’s performance. The hierarchical representation has the largest impact. Removing it reduce step-level accuracy by 10.3 on algorithm-generated trajectories and 5.0 on hand-crafted trajectories. The external prior is also critical, particularly for algorithm-generated trajectories. Without it, agent-level and step-level accuracy decrease by 10.9 and 5.5, respectively. Removing transition typing yields consistent drops across both datasets, including a 10.5 decrease in agent-level accuracy on hand-crafted trajectories. Overall, these results suggest that *FALAT* benefits from all three components, with the hierarchical representation playing the largest role.

5 Limitations

Dependence on LLM judgments. *FALAT* relies on LLM-based judgments at multiple stages, including candidate selection, transition typing, and counterfactual verification. As a result, its effectiveness depends on the underlying model’s reasoning reliability, and errors in intermediate judgments may propagate through the pipeline.

Limited long-range dependency capture. *FALAT* constructs hierarchical abstractions and typed dependencies using window-based processing, which may miss long-range dependencies that span across windows. Although consolidation partially mitigates this issue, complex interactions across distant steps may not be fully captured.

Assumption of localized failure. Our formulation assumes that failures can be localized to a small set of decisive steps under a counterfactual correction objective. In scenarios where failures arise from distributed or interacting errors across multiple steps, this assumption may not fully hold, potentially limiting the precision of attribution.

6 Related Work

LLM-based multi-agent systems. LLM-based multi-agent systems coordinate specialized agents through structured communication, role assignment, and tool use [Li et al., 2023, Wu et al., 2024, Hong et al., 2023, Qian et al., 2024]. Recent work increasingly automates the design of these systems, including prompt optimization [Khatab et al., 2023, Yuksekogonul et al., 2024], routing and model selection [Chen et al., 2025, Yue et al., 2025], adaptive interaction topologies [Zhuge et al., 2024], and workflow generation [Li et al., 2025a, Zhang et al., 2024, Hu et al., 2024, Zhang et al., 2025c,d]. These advances improve agent capability but also produce long, interdependent trajectories in which errors can propagate across agents, steps, and tool outputs. Our work focuses on diagnosing such failures by reasoning over dependencies within a single failed trajectory.

Failure attribution and trajectory analysis. Recent work has begun to study failures in multi-step LLM agent trajectories. Who&When [Zhang et al., 2025b] formalizes multi-agent failure attribution and shows that step-level localization remains challenging. Other studies analyze trajectory-level failure patterns such as loops, reasoning-action mismatches, and ineffective observation use [Bouzenia and Pradel, 2025, Banerjee et al., 2025, He et al., 2025], or introduce additional

signals for debugging, including cross-run spectrum analysis [Ge et al., 2025], implicit execution traces [Li et al., 2025b], and graph-based interaction tracing [Zhang et al., 2025e]. Most related to our setting, AgenTracer [Zhang et al., 2025a] learns an attribution model from synthetic counterfactual data, while CHIEF [Wang et al., 2026] performs backtracking over a reconstructed causal graph. In contrast, *FALAT* diagnoses a single failed trajectory without task-specific training data or oracle guidance. *FALAT* explicitly types step-to-step dependencies to separate error origination from propagation and verifies candidates through counterfactual sufficiency.

7 Conclusion

In this paper, we reformulated failure attribution in LLM agent trajectories as *hierarchical dependency-guided search* rather than step-level error classification. Based on this formulation, we proposed *FALAT*, a four-stage framework consisting of search space construction with a task-conditioned prior, candidate pruning through typed dependencies, dependency-guided search and verification, and local re-search. We evaluated *FALAT* on the Who&When benchmark, where it consistently outperformed existing SOTA baselines across multiple LLM backbones.

References

- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- Boxi Yu, Yuxuan Zhu, Pinjia He, and Daniel Kang. Utboost: Rigorous evaluation of coding agents on swe-bench. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3762–3774, 2025.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.
- Md Nakhla Rafi, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. Order matters! an empirical study on large language models’ input order bias in software fault localization. In *Proceedings of the 48th international conference on Software engineering*, 2026.
- Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiayun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, et al. Where llm agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*, 2025.
- Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. Agentracer: Who is inducing failure in the llm agentic systems? *arXiv preprint arXiv:2509.03312*, 2025a.
- Karina Zainullina, Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Daria Litvintseva, Simon Karasik, Filipp Fisin, Sergei Skvortsov, Maksim Nekrashevich, et al. Guided search strategies in non-serializable environments with applications to software engineering agents. *arXiv preprint arXiv:2505.13652*, 2025.
- Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025b.
- Yawen Wang, Wenjie Wu, Junjie Wang, and Qing Wang. From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems. *arXiv preprint arXiv:2602.23701*, 2026.
- MiniMax AI. Minimax m2.5: Built for real-world productivity. <https://www.minimax.io/news/minimax-m25>, 2025. Accessed: 2026-05-05.

- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- Earl D Sacerdoti. Planning in a hierarchy of abstraction spaces. *Artificial intelligence*, 5(2):115–135, 1974.
- Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2):181–211, 1999.
- Linqiang Guo, Wei Liu, Yi Wen Heng, Tse-Hsun Peter Chen, and Yang Wang. Agent-sama: State-aware mobile assistant. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 29459–29467, 2026.
- Islem Bouzenia and Michael Pradel. Understanding software engineering agents: A study of thought-action-result trajectories. *arXiv preprint arXiv:2506.18824*, 2025.
- Yuta Miki, Tomoko Kojiri, and Kazuhisa Seta. “if thinking” support system for training historical thinking. *Procedia Computer Science*, 60:1542–1551, 2015.
- Linxin Song, Jiale Liu, Jieyu Zhang, Shaokun Zhang, Ao Luo, Shijian Wang, Qingyun Wu, and Chi Wang. Adaptive in-conversation team building for language model agents. *arXiv preprint arXiv:2405.19425*, 2024.
- Grégoire Mialon, Clémentine Fourier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2023.
- Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. Assistantbench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8938–8968, 2024.
- Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, et al. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024.
- Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for “mind” exploration of large language model society. *Advances in neural information processing systems*, 36:51991–52008, 2023.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*, 2023.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 15174–15186, 2024.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic “differentiation” via text. *arXiv preprint arXiv:2406.07496*, 2024.

- Lingjiao Chen, Jared Quincy Davis, Boris Hanin, Peter Bailis, Matei Zaharia, James Zou, and Ion Stoica. Optimizing model selection for compound ai systems. *arXiv preprint arXiv:2502.14815*, 2025.
- Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyang Qi. Masrouter: Learning to route llms for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572, 2025.
- Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.
- Boyi Li, Zhonghan Zhao, Der-Hong Lee, and Gaoang Wang. Adaptive graph pruning for multi-agent communication. *arXiv preprint arXiv:2506.02951*, 2025a.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025c.
- Guibin Zhang, Kaijie Chen, Guancheng Wan, Heng Chang, Hong Cheng, Kun Wang, Shuyue Hu, and Lei Bai. Evoflow: Evolving diverse agentic workflows on the fly. *arXiv preprint arXiv:2502.07373*, 2025d.
- Adi Banerjee, Anirudh Nair, and Tarik Borogovac. Where did it all go wrong? a hierarchical look into multi-agent error attribution. *arXiv preprint arXiv:2510.04886*, 2025.
- Pengfei He, Zhenwei Dai, Bing He, Hui Liu, Xianfeng Tang, Hanqing Lu, Juanhui Li, Jiayuan Ding, Subhabrata Mukherjee, Suhang Wang, et al. Traject-bench: A trajectory-aware benchmark for evaluating agentic tool use. *arXiv preprint arXiv:2510.04550*, 2025.
- Yu Ge, Linna Xie, Zhong Li, Yu Pei, and Tian Zhang. Who is introducing the failure? automatically attributing failures of multi-agent systems via spectrum analysis. *arXiv preprint arXiv:2509.13782*, 2025.
- Z. Li et al. When only the final text survives: Implicit execution tracing for multi-agent attribution. *arXiv preprint*, 2025b.
- Heng Zhang, Yuling Shi, Xiaodong Gu, Haochen You, Zijian Zhang, Lubin Gan, Yilei Yuan, and Jin Huang. Graphtracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search. *arXiv preprint arXiv:2510.10581*, 2025e.

A Prompt Templates

A.1 Prompts for Search-Space Construction

External-prior prompt (π construction). This prompt operationalizes the construction of the external prior π from the task specification, agent roster, and observed incorrect output $\hat{o}$, without exposing the intermediate trajectory. Its purpose is to define the expected output type, the reasoning requirements of the task, the role boundaries of each agent, and a task-specific set of failure risks that later guide candidate selection.

```
# QUESTION
Question: {question}
Expected answer type: {answer_type}
Constraints: {constraints}

# AGENTS
{agents_block}

# AGENT'S FINAL ANSWER (WRONG)
{final_answer}

# INSTRUCTIONS
Produce the following sections. Stay observational; do not plan a solution.

===QUESTION_ANALYSIS===
- What is the question asking for? Be precise about the answer type.
- What kind of reasoning is required (e.g., retrieval, calculation, comparison,
cross-referencing, verification)?
- What makes this question hard (e.g., ambiguity, multiple required data points, domain
knowledge)?

===AGENT_ROLES===
For each agent, describe:
- What domain expertise they bring
- What they are equipped to do vs. what falls outside their stated role
- What a competent agent in this role should be expected to catch or verify

===ERROR_SIGNALS===
Based on the question type and the observed wrong final answer, what patterns should we
watch for when reading the trajectory?
Examples of error signals:
- Data truncation (incomplete lists, pagination, "showing  $N$  of  $M$ ")
- Cross-reference with incomplete data on one or both sides
- Fabricated data (values that appear without a retrieval or calculation source)
- Domain confusion (agent working outside its expertise)
- Verification theater (agent "confirms" by restating, not by checking)
- Wrong entity / wrong time period / off-by-one errors
- Placeholder values used as real data

List 3-5 specific signals tailored to this question.
```

Hierarchical-representation prompt (M construction). This prompt constructs the hierarchical representation $M = (M_{hi}, M_{mid}, M_{lo})$ conditioned on the external prior π . It organizes the observed trajectory into coarse execution summaries, local step summaries, and low-level evidence, which together define the initial search space.

```
# CONTEXT ANALYSIS (your lens for reading this trajectory)
{context_analysis}

# TASK CONTEXT
Question: {question}
Expected answer type: {answer_type}
Constraints: {constraints}

Agents involved:
{agents_block}

Agent's final answer: {final_answer}
```

```

Step-to-Agent index ({num_steps} steps total):
{step_index}

# FULL TRAJECTORY ({num_steps} steps)
{steps_block}

# INSTRUCTIONS
Using the context analysis as your lens, read the trajectory and produce three memory
documents. Focus on what is surprising, suspicious, or inconsistent, not on confirming
expectations.

===HIGH_LEVEL===
Write 4-6 bullet points covering:
- What is the task asking?
- What did the agents answer?
- What overall strategy did the agents follow?
- What looks wrong at a glance?
- Which agent handled the most critical work?

===MID_LEVEL===
Write one entry per key moment. Skip all routine steps.
For each key moment:
### Step N (AgentName)
What happened and why it matters. (2-3 sentences max)
Include a step if:
- It computed or chose a value that feeds into the final answer
- It made a decision that changed the trajectory's direction
- Something looks wrong, suspicious, or surprising
- It introduced NEW information
- It matches an error signal from the context analysis

===LOW_LEVEL===
Write one entry per suspicious or critical step.
For each entry:
### Step N (AgentName)
- Exact values computed or used
- Data provenance
- Data completeness
- Assumptions made
- What specifically looks wrong or risky
- What inputs this step received and whether they were correct

```

A.2 Prompts for Typed Dependency Construction

Typed-dependency prompt. This prompt operationalizes typed dependency construction over di-agnostically salient steps. It uses the hierarchical representation M to assign transition labels, detect loops and dead ends, and induce a value-flow structure that is later used to prune candidates from $\mathcal{C}$ to $\hat{\mathcal{C}}$.

```

# HIERARCHICAL REPRESENTATION
## High Level
{high_level}

## Candidate Steps / Key Moments
{mid_level}

## Supporting Evidence
{low_level}

# STEP INDEX (all {num_steps} steps, condensed)
{step_index}

# INSTRUCTIONS
Analyze how the candidate steps depend on one another. Your goal is to determine which
candidates may have introduced, propagated, transformed, or eliminated a deviation that
affects the final output.
For each ordered pair of candidate steps in temporal order:
1. Determine whether the later step depends on the earlier step's output, decision, or
assumption.

```

```

2. Assign one transition label from the set below.
3. State briefly how the later step relates to the earlier step and whether this
relation suggests source, propagation, correction, redundancy, or irrelevance.

# OUTPUT FORMAT
Write a single markdown document with the following sections:

## Typed Dependencies
For each relevant ordered pair:
### Step X → Step Y
- Type: follow_up | redundancy | no_influence | error_shift | correction | dead_end
- Relationship: 1 sentence on how Y depends on X.

# TRANSITION LABELS
- follow_up: step Y builds on the output, decision, or assumption introduced by step
X. If step X is already corrupted, treat step Y as a downstream carrier.
- redundancy: step Y repeats, restates, or rechecks information already provided by
step X without adding new evidence or changing the trajectory.
- no_influence: step Y does not consume or depend on the output of step X.
- error_shift: step Y introduces a new deviation, shifts the trajectory away from the
expected path, or transforms an upstream issue at step X into a different downstream
error.
- correction: step Y fixes, compensates for, or invalidates a deviation introduced by
step X.
- dead_end: step X leads to an output that is not consumed by step Y or by the final
answer.

## Candidate-Pruning Summary
Summarize which candidates remain plausible error sources or downstream carriers
whose effects can still reach the final output, and which candidates can be pruned as
redundant, corrected, unrelated, or dead ends.

## Value Flow
Describe how information and decisions propagate from retained candidates toward the
final output. If the chain is incomplete or broken, state where and why.

```

A.3 Prompts for Dependency-Guided Search

High-confidence candidate selection prompt (C_H). This prompt narrows the refined candidate set C' to a small high-confidence subset C_H using the external prior, the typed dependency structure, and the compressed trajectory representation.

```

# TASK CONTEXT
Question: {question}
Expected answer type: {answer_type}
Final answer given: {final_answer}

Agents:
{agents_block}

Step-to-Agent mapping:
{step_index}

# TRAJECTORY REPRESENTATION
## Big Picture
{high_level}

## Key Moments
{mid_level}

{transitions_section}

# INSTRUCTIONS
Identify the 2-3 steps most likely to contain the decisive error. Use the step-to-agent
mapping to assign the correct agent to each step. Rank the candidates by likelihood,
with the most suspicious step first.

Output only the JSON object.

```

Candidate verification prompt. This prompt verifies candidates in $\mathcal{C}_H$ against the counterfactual sufficiency objective using localized raw evidence, typed dependencies, and linked low-level evidence.

```
# TASK CONTEXT
Question: {question}
Final answer given: {final_answer}

Step-to-Agent mapping:
{step_index}

# CANDIDATES TO VERIFY
{candidates_block}

# LOCAL RAW EVIDENCE
Only the candidate steps and their immediate neighbors are shown. Candidate steps are
marked explicitly.
{raw_steps}

# LINKED LOW-LEVEL EVIDENCE
{low_level}

# INSTRUCTIONS
For each candidate, evaluate the following three criteria:
1. Error independence. Does this step introduce the error, or merely inherit it from an
upstream fault? Use the typed dependency structure to trace whether the candidate is an
error source or a downstream carrier.

2. Counterfactual sufficiency. If this step's output, decision, or assumption were
corrected, would the final answer recover the expected output while keeping prior steps
fixed and allowing subsequent steps to adapt?

3. Temporal precedence. Among candidates satisfying the above criteria, prefer the
earliest step.

Select the candidate that best satisfies these conditions and return the decisive error
together with supporting reasoning.

Output only the JSON object.
```

A.4 Prompts for Local Re-Search

Local re-search prompt (adversarial verification). This prompt performs bounded local re-search around the predicted decisive step $\hat{t}$, comparing it directly against neighboring candidates under an adversarial for-and-against evaluation.

```
# TASK CONTEXT
Question: {question}
Final answer given: {final_answer}
Agents: {agents_block}
Total steps: {total_steps}

# PRELIMINARY DIAGNOSIS
Predicted decisive error: Step {t} by {a} -- {predicted_desc}
Backward chain: {backward_chain}

# LOCAL WINDOW ( $[\hat{t} - 2, \hat{t} + 2]$ )
{candidates_block}

# ROLE DEFINITIONS
For each step  $X$  in the local window, assign one of the following roles:
- ROOT_CAUSE: step  $X$  independently introduces the decisive error, such that correcting
it would change the final output.
- PROPAGATION: step  $X$  consumes an upstream error and carries it into downstream
reasoning or output.
- SYMPTOM: step  $X$  reflects an existing error without materially contributing to
downstream computation.
- CONTRIBUTING: step  $X$  is independently problematic but is not by itself sufficient to
change the final output.
```



```

# INSTRUCTIONS
## Step A: FOR-AND-AGAINST
For each step in the local window:
1. give 1-2 arguments for it being the decisive error;
2. give 1-2 arguments against it being the decisive error;
3. assign a suspicion score in [0,1];
4. assign one role: ROOT_CAUSE, SYMPTOM, PROPAGATION, or CONTRIBUTING.
Be critical. Do not simply confirm the current diagnosis; challenge it against nearby alternatives.

## Step B: COUNTERFACTUAL TEST
For the top-3 candidates by suspicion score:
- If only this step were corrected, would the final answer become correct?
- Provide one sentence of reasoning.

## Step C: FINAL VERDICT
Based on Steps A and B, select the step with the highest suspicion score that also passes the counterfactual test. Produce a ranked list of all local candidates and return either CONFIRM or REVISE.

# OUTPUT FORMAT
{
  "candidates": [
    {
      "step": <int>,
      "agent": "<name>",
      "for": ["<point>", ...],
      "against": ["<point>", ...],
      "suspicion_score": <float 0.0-1.0>,
      "classification": "<ROOT_CAUSE | SYMPTOM | PROPAGATION | CONTRIBUTING>"
    }
  ],
  "counterfactual_tests": [
    {
      "step": <int>,
      "passes": <true | false>,
      "reasoning": "<1 sentence>"
    }
  ],
  "ranked_steps": [<int>, <int>, ...],
  "final_verdict": {
    "decision": "<CONFIRM | REVISE>",
    "decisive_step": <int>,
    "decisive_agent": "<name>",
    "failure_mode": "<wrong_query | wrong_logic | wrong_interpretation | wrong_assumption | wrong_calculation | wrong_filter | missed_info | hallucination | other>",
    "description": "<1-2 sentences>",
    "confidence": "<low | moderate | high>",
    "reasoning": "<1-2 sentences on why you confirmed or revised>"
  }
}

Output only the JSON object.

```

B Compute and Execution Setup

All experiments are conducted using API-based inference. We use the OpenAI API and OpenRouter to access the underlying LLM backbones. No local model training or fine-tuning is required. The experiments can be reproduced by invoking the same APIs with the specified prompts, model configurations, and parameters described in this paper. Execution time depends on API latency and the number of LLM calls per trajectory, as *FALAT* involves multiple stages (prior construction, hierarchical abstraction, dependency typing, and verification).